PG Finder Agent - Project Audit and Completion Plan

Prepared for: WildCatter Ventures Project Assignment **Audit date:** 20 October 2026 **Assignment deadline:** Tuesday, 20 October 2026 **Project:** AI-native student PG accommodation finder

Executive verdict

This is a strong-looking and genuinely agentic prototype, but it should not be submitted unchanged. The UI, tool timeline, natural-language parsing, map, enquiry drafting, and calendar flow create a convincing product demonstration. The main weakness is data integrity: when grounded web search is unavailable, the application can accept listing details produced from ungrounded model knowledge and still present them as live public-source results.

The exact assignment request completed successfully:

> Please find me a PG accommodation near Mithibai college within 1 km radius. My budget is Rs 5,000 and I need the accommodation in 5 days.

The application parsed the radius, budget, landmark, and deadline correctly, displayed the search activity, rendered a map, ranked a result, drafted an enquiry, and produced calendar links. However, the returned "Sai Krupa PG Accommodation" could not be reliably corroborated as a Mithibai-area listing and was plotted at exactly the college coordinates, resulting in an implausible 0 metre distance.

The right submission strategy is not to redesign the app. The visual layer is already one of its strengths. The priority is to make the data pipeline honest, fix the broken demo interactions, add defensible tests, and deploy a clean build.

Audit health score

Dimension	Score	Key finding
Accessibility	2/4	Inputs have labels, but keyboard focus is invisible, live progress is not announced, and many controls are smaller than 44 px.
Performance	3/4	Production build is healthy; the map and repeated glass/blur effects are moderately heavy.
Responsive design	2/4	The homepage works on mobile, but the results view expands to 446 px on a 375 px viewport.
Theming	2/4	The dark visual system is coherent, but design tokens and numerous hard-coded utility colours are mixed.
Implementation integrity	1/4	Ungrounded model output can be labelled and cached as live listing data.
Total	**10/20**	**Acceptable - significant work needed before submission**

Approximate issue count: **1 P0, 7 P1, 4 P2, and 2 P3 issues**.

What is already working well

- The product is genuinely AI-native. A server-side Gemini agent chooses tools, receives function responses, streams activity through Server-Sent Events, and has a deterministic fallback planner.
- The exact assignment sentence works from input to results.
- Text input and browser-supported Web Speech voice input are implemented.
- Parsed parameters are visible and editable, including landmark, radius, budget, and move-in date.

- Unknown prices are separated into a clear "Worth asking about" group instead of being fabricated or silently discarded.
- Listings are labelled unverified and availability is not claimed as guaranteed.
- The application avoids fake room photography and exposes source badges.
- Enquiry drafting, copy-to-clipboard, source links, `.ics` generation, and Google Calendar links work after manual interaction.
- The calendar wording correctly states that a visit is proposed and not confirmed by an owner.
- The desktop interface is attractive, coherent, and easy to demonstrate.
- The production Next.js build passes.
- The production dependency audit reported zero known npm vulnerabilities.
- Secrets remain server-side and `.env` is ignored by Git.

Evidence gathered during the audit

The following checks were performed against the current working tree:

- `npm run build`: passed; all pages and API routes compiled.
- `npm run lint`: failed with 14 errors and 7 warnings.
- Exact Mithibai request: completed with HTTP 200 and one result.
- Browser console: no application JavaScript errors during the tested flow.
- Contact draft endpoint: HTTP 200.
- Visit proposal endpoint: HTTP 200; `.ics` and Google Calendar actions appeared.
- Desktop viewport: no horizontal overflow.
- Mobile homepage at 375 x 667: no horizontal overflow.
- Mobile results at 375 x 667: document width reached 446 px.
- Keyboard test: the first focused textarea had no visible outline or box shadow.
- Production dependency security audit: zero known vulnerabilities.
- Git state: 17 modified files and 4 untracked files; no Git remote was configured.

P0 blocking issue

P0 - Ungrounded model output is presented as live listing data

Location: `src/lib/webDiscovery.ts`, especially the fallback beginning around line 129.

When Google Search grounding fails or its quota is exhausted, the code asks Gemini for listing data without live search access. It then accepts the returned business name, amenities, source name, source URL, and other claims as listing data. Those records can be geocoded, cached for 24 hours, ranked, and displayed under the label "live public sources."

The local server logged that grounding quota was exhausted and that the knowledge fallback was used. The resulting "Sai Krupa PG Accommodation" linked to a generic Justdial category page, not an exact business listing. An independent check of the category page showed other businesses in Vile Parle, while the directly identifiable Sai Krupa PG result was located in Chennai.

References:

- Justdial category for men's PGs in Vile Parle West
- Sai Krupa PG Accommodation result in Chennai

Impact: The central product promise - that the agent finds real accommodation - becomes unreliable. This is more serious than a UI bug because it can mislead a student into acting on a place that may not exist in the claimed location.

Required fix:

1. Remove the ungrounded model fallback for listings.
2. If grounded web discovery fails, return only OSM/Geoapify data or clearly state that live web discovery is unavailable.
3. Require an exact listing or business-detail URL. Reject homepages, category pages, and generated Google search URLs.
4. Record provenance for each field: source URL, extraction time, grounded citation, and validation state.
5. Do not cache records that have not passed validation.
6. Display a source-status indicator such as `Grounded`, `Directory POI`, `Curated pilot`, or `Unverified lead`.

Fastest safe alternative for the deadline

Narrow the submission to a **Mithibai pilot**. Create a small dataset of 8-15 manually verified accommodation leads with exact URLs, real coordinates, last-checked timestamps, and honest unknown values. Let the AI agent do the natural-language parsing, filtering, ranking, explanation, enquiry drafting, and visit planning.

This still satisfies the AI-native requirement and is much more defensible than claiming unreliable nationwide live coverage.

P1 major findings

1. Listing geocoding can collapse onto the landmark

Location: `src/lib/geo.ts` around lines 48-61 and 80-85; `src/lib/webDiscovery.ts` around lines 161-179.

Curated landmark matching checks whether words such as "Mithibai" occur anywhere in a geocoding query. A listing query that includes "near Mithibai College" can therefore inherit the college's curated coordinates. This produced the 0 metre result.

Fix: Use curated coordinates only for the user's requested landmark. Listing geocoding must use a verified street address or source-provided coordinates. Reject results that resolve to the exact landmark coordinate unless the source explicitly identifies accommodation at that location.

2. Product claims exceed actual behavior

Location: `README.md` near the opening claim and `src/app/page.tsx` around line 218.

The README claims "100% real live public data." The homepage says the agent "messages owners and books a visit." In reality, it drafts a message for the student to send and creates an unconfirmed calendar event.

Recommended replacement copy:

> The agent finds and ranks public accommodation leads, drafts outreach, and proposes visit times. Students confirm price, availability, identity, and visits directly with the owner.

3. The generated enquiry does not appear in the result card

Location: `src/components/ResultCard.tsx` around line 63.

The component copies `initialDraft` into local state only during its first render. The card is mounted before the streamed enquiry event arrives, so later prop updates are ignored. The activity timeline says the enquiry was drafted, but the card still asks the user to draft it again.

Fix: Either render directly from the draft prop or synchronize it with `useEffect`. Add an integration test that waits for the streamed draft and confirms the copy/source/WhatsApp controls appear without a second request.

4. Mobile results horizontally overflow

Location: `src/app/page.tsx` around line 314.

The results header's flexible search container does not shrink. At a 375 px viewport its right edge reached 446 px.

Fix: Add `min-w-0` to the glass search container, make the header responsive, hide or shorten the brand subtitle on narrow screens, and verify at 320, 375, 390, and 430 px.

5. Accessibility is incomplete

Locations: main and agent inputs in `src/app/page.tsx`, editable chips in `src/components/ParamChips.tsx`, and the activity timeline in `src/components/Timeline.tsx`.

Problems observed:

- The main textarea deliberately removes its outline and has no replacement focus ring.
- Dynamic activity and final results are not exposed through `aria-live` or an equivalent status region.
- The 32 px agent microphone and several 16-39 px actions are below the recommended 44 px touch target.
- Motion has no `prefers-reduced-motion` alternative.
- Several labels and metadata elements use 10-11 px text.

Fix: Add consistent `focus-visible` rings, a polite live region for progress, a more assertive error region, 44 px action targets, minimum 12 px secondary text, and reduced-motion variants.

6. The test suite gives false confidence

Location: `scripts/test_queries.ts` around lines 78-90 and 92-142.

The script reads source statistics from the ranked event even though the application emits them in a separate `source_stats` event. This is why every saved `sourceStats` object is empty. The honesty test only checks that a record is not sample data; it does not verify the business name, exact source URL, coordinates, source contents, or grounded provenance.

Fix:

- Read the actual `source_stats` event.
- Fail when source statistics are missing.
- Reject generic category/homepage URLs.
- Assert distance and coordinates are plausible.
- Test grounded-search failure and quota exhaustion.
- Add fixtures for deterministic parsing, ranking, deduplication, and enquiry generation.

7. Quality gate and repository state are not submission-ready

`npm run lint` currently reports 14 errors and 7 warnings, including unsafe `any` usage, unused state/imports, and unescaped JSX quotation marks. There are also 17 modified and 4 untracked project files, while no Git remote is configured.

Fix: Make lint pass, review the entire diff, commit the source, push it, and test from a clean clone.

P2 secondary findings

API protection and validation

The public agent endpoint has no rate limit, request schema validation, or abuse control. A deployed API key could be exhausted by repeated calls.

Add a schema validator, per-IP/session throttling, maximum request duration, and structured error codes. Keep model/key failure details out of production user messages.

Runtime cache durability

Nominatim, Overpass, and web discovery caches write synchronously to the local project filesystem. This is best-effort and may not persist reliably on serverless hosting.

For the MVP, treat caches as optional and ensure correctness never depends on them. For production, use a durable store such as Redis, KV, or a small database.

Development startup changes tracked demo data

The `predev` script runs the seed generator and rewrites tracked `data/listings.json` every time development starts. This creates noisy diffs and makes it harder to distinguish intentional changes.

Remove seeding from `predev`, make seed output deterministic, or write generated data to an ignored runtime location.

Voice support is browser-dependent

The Web Speech API implementation has a good typing fallback, but speech recognition is not consistently available across browsers. The UI should explicitly recommend Chrome/Edge for voice demos and always keep text input as the reliable path.

P3 polish findings

The mechanical design detector found four warnings: decorative gradient text, two cases of grey text on coloured backgrounds, and a familiar purple/pink AI-style palette. These are not blockers because the interface is coherent, but they can make the product feel template-generated.

If time remains, use a more product-specific visual identity connected to student housing and Mumbai navigation. Do not prioritize this above data integrity, testing, or deployment.

Definition of a complete submission

A credible submission should include:

1. A live deployed URL that works without local setup.
2. A GitHub repository with a clean working tree and reproducible setup.
3. The exact assignment prompt prefilled as the primary demo example.
4. Grounded or manually verified accommodation leads with precise provenance.
5. Honest messaging when no verified option satisfies Rs 5,000 within 1 km.
6. A visible agent timeline showing parse, geocode, search, rank, draft, and calendar tools.
7. A working text flow and a graceful voice-input fallback.
8. Responsive desktop and mobile layouts.
9. Passing production build, lint, and automated tests.
10. A concise README explaining architecture, setup, limitations, and evaluation evidence.
11. A 60-90 second demo video showing the exact prompt, one source check, enquiry drafting, and proposed calendar visit.

Deadline-focused completion plan

Phase 1 - Data integrity (60-90 minutes)

- Remove the ungrounded listing fallback.
- Choose either a verified Mithibai pilot dataset or grounded-only results.

- Fix listing geocoding.
- Add provenance/status to every result.
- Correct README and homepage claims.

Phase 2 - Demo journey (60 minutes)

- Repair the enquiry draft state synchronization.
- Fix the mobile header overflow.
- Add visible focus rings and a live status region.
- Increase important mobile touch targets.
- Re-test the exact assignment prompt.

Phase 3 - Trustworthy tests (45 minutes)

- Make lint pass.
- Fix source-stat collection.
- Add tests for parsing, quota failure, coordinate plausibility, exact URLs, and draft rendering.
- Run the build and tests from a clean terminal.

Phase 4 - Package and deploy (45 minutes)

- Review and commit all changes.
- Create and push to a GitHub remote.
- Deploy to Vercel with environment variables configured.
- Run the exact request on the deployed URL.
- Record the short demonstration video.

Recommended implementation order

1. **P0 - Harden data provenance:** remove ungrounded web listings and reject generic URLs.
2. **P1 - Correct claims:** rewrite homepage and README language.
3. **P1 - Fix functional bugs:** geocoding, enquiry state, and mobile overflow.
4. **P1 - Establish quality gates:** clean lint and add meaningful tests.
5. **P1 - Deploy:** clean commit, GitHub remote, hosted URL, production verification.
6. **P2 - Accessibility and resilience:** focus, live regions, touch targets, rate limits, and durable cache strategy.
7. **P3 - Final polish:** refine typography, palette, and product-specific visual identity.

Final recommendation

The project has enough substance to become a strong submission. Its agent architecture and UI are already more ambitious than a conventional filter form. The fastest path to credibility is to reduce scope and increase truthfulness: a narrow, verified Mithibai experience that always tells the user what is known, unknown, and unavailable will score better than a nationwide system that can invent leads when a free search quota fails.

Do not submit the current "100% real live data" claim. Fix the integrity path first, then commit, deploy, and demonstrate the exact assignment prompt.